

Technische Informatik

Lernskript zur Vorbereitung auf die mündliche Prüfung

Stand: 11.6.2026

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Einordnung & Grundbegriffe	4
Roter Faden für die Prüfung	4
2. Rechnerarchitektur & CPU	4
Aufgaben der CPU-Bestandteile	4
Befehlszyklus	5
3. Speicher, Busse & I/O	5
Speicherarten und Begriffe	5
Little-Endian und Big-Endian	6
Busse und I/O	6
4. Architekturmodelle, RISC/CISC & Pipeline	6
Von-Neumann und Harvard	6
RISC und CISC	8
Pipeline und Konflikte	8
5. Mikrocontroller & ATmega328P / Arduino Uno	8
ATmega328P - wichtige Merkmale	8
AVR-Speicheraufteilung	9
6. AVR-Assembler: Programmaufbau & Register	9
Direktive vs. Kommando	9
Typischer AVR-Programmaufbau	9
Registerempfehlungen	10
7. AVR-I/O-Ports: DDRx, PORTx, PINx	10
Wichtige I/O-Befehle	11
Pull-up/Pull-down und Taster	11
8. Interrupts, Stack & Unterprogramme	11
Ablauf eines AVR-Interrupts	11
Voraussetzungen	11
Stack	12
Unterprogramme und Makros	12
9. Zeitsteuerung, Timer & PWM	13
Warteschleifen	13
Timer0-Modi	13
Timer-Berechnung	13
PWM	13
10. USART, ADC & EEPROM	14
USART	14
ADC und EEPROM	14
I2C	14
11. Intel 8086 / x86-Grundlagen	15
Assembler allgemein	15
Warum Assembler?	15

Programmstruktur unter DOS/TASM	15
12. x86-Register, Flags & Segmentierung	16
8086-Registergruppen	16
Wichtige Flags	16
Segmentierung und Adressrechnung	16
13. DOS-Interrupts, Schleifen, Prozeduren & Makros	17
DOS-Interrupt 21h	17
Sprünge und Schleifen	17
Prozeduren, Stack und Parameter	17
Makros vs. Prozeduren	18
14. Prüfungsfragen & Kurz-Wiederholung	18
Kompakte Prüfungsfragen	18
Formeln und Mini-Merksätze	19

1. Einordnung & Grundbegriffe

Technische Informatik verbindet Informatik mit der konkreten Hardware. Du sollst nicht nur wissen, was ein Programm macht, sondern auch, wie Befehle, Register, Speicher, Busse, Interrupts und Peripherie zusammenarbeiten.

Begriff	Bedeutung
Rechnerarchitektur	Struktur, Design und Organisation eines Rechners: Welche Komponenten gibt es und nach welchen Prinzipien arbeiten sie?
Mikroprozessor	CPU auf einem Chip; enthält Rechenwerk, Steuerwerk, Register und interne Busse.
Mikrocontroller	Mikroprozessor plus Speicher und Peripherie auf einem Chip; typisch für eingebettete Systeme.
Assembler	Maschinennahe Sprache aus Mnemonics; der Assembler übersetzt sie in Maschinencode.
Befehlssatz	Menge der Maschinenbefehle, die eine CPU versteht; unterscheidet z. B. RISC und CISC.
Peripherie	Schnittstellen zur Außenwelt: Taster, LEDs, Sensoren, Timer, UART, ADC, Speicher usw.

Roter Faden für die Prüfung

1. **Vom Bit zur Hardware:** 0/1 werden über Schalter/Transistoren und logische Gatter realisiert.
2. **Von Hardware zum Befehl:** Register, ALU und Steuerwerk führen Maschinenbefehle aus.
3. **Vom Befehl zum Programm:** Assembler schreibt diese Befehle symbolisch und strukturiert.
4. **Vom Programm zur Peripherie:** I/O-Register, Timer, Interrupts und Schnittstellen verbinden Programm und Außenwelt.



In der mündlichen Prüfung wirkt es sehr gut, wenn du immer die Kette erklärst: Hardwarezustand -> Register/Befehl -> Programmablauf -> Wirkung an einem Pin oder Gerät.

2. Rechnerarchitektur & CPU

Ein Rechner besteht aus CPU, Speicher, Ein-/Ausgabeeinheiten und einem Bussystem. Die CPU verarbeitet Daten und koordiniert rechnerinterne Aktionen. Sie besteht im Kern aus Steuerwerk, Rechenwerk/ALU, Registern und internen Verbindungen.

Aufgaben der CPU-Bestandteile

Bestandteil	Aufgabe
ALU / Rechenwerk	Führt arithmetische und logische Operationen aus: Addition, Subtraktion, Vergleiche, AND, OR, XOR, Bitverschiebungen.
Steuerwerk	Holt Befehle, decodiert sie und erzeugt die Steuersignale für Register, ALU, Speicher und I/O.

Registersatz	Sehr schneller CPU-interner Speicher für Operanden, Ergebnisse, Adressen, Status und Steuerinformationen.
Program Counter / IP	Enthält die Adresse des nächsten Befehls. Wird beim Befehlsabruf fortgeschrieben oder durch Sprünge verändert.
Stackpointer	Zeigt auf den Stack; wichtig für Unterprogramme, Rücksprungadressen, Interrupts und temporäre Daten.
Flagregister	Speichert Statusbits aus der letzten Operation, z. B. Zero, Carry, Overflow, Sign oder Parity.

Befehlszyklus

5. **Instruction Fetch:** Befehl aus dem Speicher holen; PC/IP zeigt auf die Adresse.
6. **Decode:** Befehl interpretieren: Welche Operation? Welche Operanden?
7. **Operand Fetch:** Benötigte Operanden aus Registern, Speicher oder I/O holen.
8. **Execute:** Operation in ALU/Steuerwerk ausführen.
9. **Write Back:** Ergebnis zurück in Register/Speicher schreiben und Flags aktualisieren.



Die ALU rechnet, das Steuerwerk organisiert, Register halten kurzfristig Daten und Flags beschreiben das Ergebnis.

3. Speicher, Busse & I/O

Speicher stellt Informationen bereit: Programmcode, Daten und Informationen über den Programmablauf. Je näher ein Speicher an der CPU liegt, desto schneller und teurer ist er normalerweise.

Speicherarten und Begriffe

Speicher	Eigenschaft	Prüfungsrelevanz
Register	Direkt in der CPU, sehr schnell, sehr klein.	Operanden, Adressen, Rückgabewerte, Status.
Cache	Schneller Zwischenspeicher zwischen CPU und RAM.	Temporalität und Lokalität erklären.
RAM	Flüchtiger Hauptspeicher, adressierbar durch CPU.	Programme/Daten während der Ausführung.
Flash / SSD	Nichtflüchtiger elektronischer Massenspeicher.	Schnellere Zugriffe als HDD, begrenzte Schreibzyklen.
HDD	Magnetische rotierende Scheiben.	Mechanische Zugriffszeiten, hohe Kapazität.
EEPROM	Nichtflüchtiger Speicher im Mikrocontroller.	Daten bleiben nach Abschalten erhalten, Schreibzyklen begrenzt.

Little-Endian und Big-Endian

Bei mehrbyteigen Werten muss festgelegt werden, in welcher Reihenfolge die Bytes im Speicher liegen. x86 verwendet Little-Endian: Das niederwertigste Byte liegt an der kleinsten Adresse. Netzwerkprotokolle verwenden häufig Big-Endian.

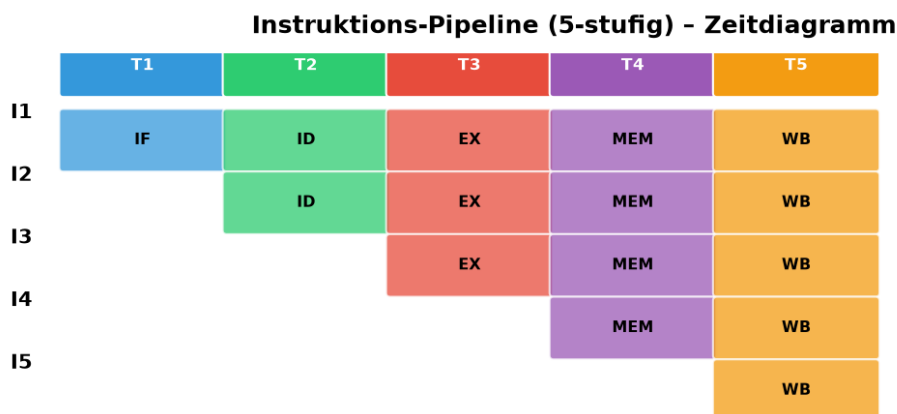
Wert 0x12345678	Adresse 1000	Adresse 1001	Adresse 1002	Adresse 1003
Little-Endian	78	56	34	12
Big-Endian	12	34	56	78

Busse und I/O

Bus	Aufgabe
Adressbus	Legt fest, welche Speicher- oder I/O-Adresse angesprochen wird.
Datenbus	Transportiert die eigentlichen Daten.
Steuerbus	Koordiniert Lesen/Schreiben, Takt, Interrupts und Richtung der Übertragung.

I/O-Einheiten bilden die Schnittstelle zur Außenwelt. Sie wandeln externe Signale in elektrische Signale und stellen Register bereit, über die die CPU oder der Mikrocontroller sie steuert.

4. Architekturmodelle, RISC/CISC & Pipeline



Ohne Pipeline: 5 Instruktionen × 5 Stufen = 25 Takte | Mit Pipeline: 5 + 4 = 9 Takte

Abb.: Instruktions-Pipeline Zeitdiagramm

Von-Neumann und Harvard

Von-Neumann vs. Harvard-Architektur

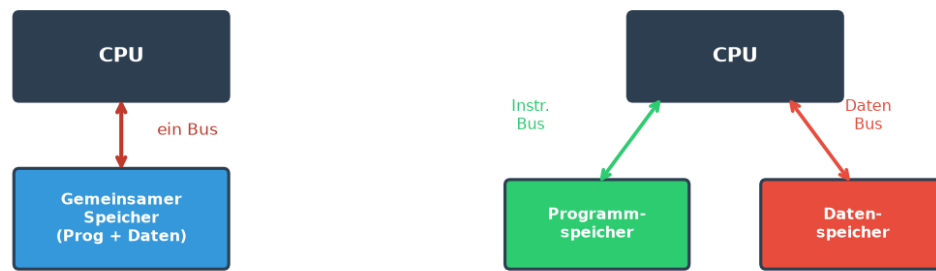


Abb.: Von-Neumann vs. Harvard-Architektur

Bei der Von-Neumann-Architektur liegen Programm und Daten im gleichen Speicher und werden über gemeinsame Wege übertragen. Bei der Harvard-Architektur sind Programm- und Datenspeicher getrennt, wodurch paralleler Zugriff möglich ist. AVR-Mikrocontroller nutzen eine Harvard-artige Trennung von Programm- und Datenspeicher.

Modell	Merkmal	Vorteil	Nachteil
Von Neumann	Gemeinsamer Speicher für Code und Daten.	Einfach, flexibel.	Von-Neumann-Flaschenhals: gemeinsamer Datenweg kann bremsen.
Harvard	Getrennter Code- und Datenspeicher mit getrennten Wegen.	Parallelität, oft schneller/robuster bei MCs.	Komplexere Organisation; Programm- und Datenzugriff verschieden.

RISC und CISC

Kriterium	RISC (z. B. AVR/ARM)	CISC (z. B. Intel x86/8086)
Befehlssatz	Reduziert, eher einfache Befehle.	Umfangreicher, komplexere Befehle.
Befehlslänge	Häufig eher konstant/regelmäßig.	Variabel und historisch gewachsen.
Ausführung	Viele Befehle in wenigen Takten, gut für Pipeline.	Befehle können unterschiedlich lange dauern.
Programmieridee	Viele einfache Schritte.	Komplexere Einzelbefehle möglich.

Pipeline und Konflikte

Pipelining überlappt Befehlsphasen: Während ein Befehl ausgeführt wird, wird der nächste schon geholt. Dadurch steigt der Durchsatz, aber es entstehen Konflikte.

Konflikt	Problem	Gegenmaßnahme
Datenkonflikt	Ein Befehl braucht ein Ergebnis, das noch nicht fertig ist.	Forwarding, Wartezyklen, Out-of-Order Execution.
Kontrollkonflikt	Bei Sprüngen ist unklar, welcher Befehl als nächstes kommt.	Branch Prediction, spekulative Ausführung.
Strukturkonflikt	Zwei Befehle brauchen dieselbe Hardwareeinheit.	Mehr Ressourcen, Scheduling.



RISC/CISC nicht als 'gut/schlecht' darstellen. Erkläre die Entwurfsphilosophie: einfache, regelmäßige Befehle gegen umfangreiche, komplexe Befehle.

5. Mikrocontroller & ATmega328P / Arduino Uno

Ein Mikrocontroller ist für Steuerungs- und Regelungsaufgaben gebaut. Er kombiniert CPU, Speicher und Peripherie in einem Baustein. Der Arduino Uno ist eine Mikrocontroller-Platine mit einem AVR ATmega328P.

ATmega328P - wichtige Merkmale

Merkmal	Wert / Bedeutung
Architektur	8-Bit-RISC-Mikrocontroller.
Flash	32 kByte Programmspeicher, in Application- und Bootloaderbereich unterteilt.
SRAM	2 kByte Datenspeicher; enthält Registerbereich, I/O-Bereich und Nutzdaten.
EEPROM	1 kByte nichtflüchtiger Datenspeicher.
I/O	23 digitale I/Os und 6 analoge Eingänge.
Timer	Zwei 8-Bit-Timer und ein 16-Bit-Timer.
Schnittstellen	USART, SPI, ADC, Analog Comparator, Watchdog, interne/externe Interrupts.
Takt	0 bis 16 MHz, Arduino Uno typischerweise 16 MHz.

AVR-Speicheraufteilung

Bereich	Bedeutung
Programmspeicher	Enthält Maschinenbefehle; bei AVR getrennt vom Datenspeicher. Zugriff auf Konstanten per Z-Zeiger und LPM.
Daten-/SRAM-Bereich	Arbeitsregister, I/O-Register, erweiterte I/O-Register und eigentliche SRAM-Daten.
Register R0-R31	32 General Purpose Register, je 8 Bit. X, Y, Z bestehen aus Registerpaaren.
Statusregister SREG	Flags wie I, T, H, S, V, N, Z, C.
Stack	Liegt im SRAM, wächst nach unten; Stackpointer muss initialisiert werden.

Beim AVR ist ein Pin nicht einfach nur 'an oder aus'. Entscheidend sind DDRx für die Richtung, PORTx für Ausgangswert/Pull-up und PINx zum Lesen des tatsächlichen Pinzustands.

6. AVR-Assembler: Programmaufbau & Register

Assembler besteht aus Direktiven und Befehlen. Direktiven steuern den Assembler, Befehle werden zu Maschinenbefehlen für den Mikrocontroller.

Direktive vs. Kommando

Art	Bedeutung	Beispiele
Direktive	Anweisung an den Assembler; steuert Übersetzung, Namen, Konstanten, Speicherbereiche.	.include, .def, .equ, .set, .org, .db, .macro
Kommando/Befehl	Anweisung an die CPU; entspricht einem Maschinenbefehl.	LDI, MOV, ADD, SUB, OUT, IN, SBI, CBI, RJMP
Kommentar	Erklärung für Menschen, nicht Teil des Maschinencodes.	; Kommentar oder /* ... */

Typischer AVR-Programmaufbau

10. **Dateikopf:** Titel, Aufgabe, Autor, Hardwarehinweise.
11. **Geräte-datei einbinden:** .include mit Definitionen für Register und Bits.
12. **Namen vergeben:** .def für Register, .equ für Konstanten.
13. **Interruptvektortabelle:** Reset- und Interruptvektoren mit rjmp auf Routinen.
14. **Initialisierung:** Stack, Ports, Timer, USART, Interruptfreigaben.
15. **Hauptschleife:** Endlosschleife oder Hauptlogik.
16. **ISR/Unterprogramme/Makros:** Strukturierung und Wiederverwendung.

Registerempfehlungen

Register	Typische Nutzung
R0	Für LPM und spezielle Operationen reservieren.
R1-R14	Allgemeine Arbeitsregister.
R15	Gut geeignet zum Sichern des Statusregisters.
R16-R23	Konstanten laden, Bitmasken, Auswertungen; viele Immediate-Befehle nur hier möglich.
R24/R25	16-Bit-Zähler; ADIW/SBIW arbeiten auf Registerpaaren.
R26/R27 = X	16-Bit-Zeigerregister für SRAM-Zugriffe.
R28/R29 = Y	16-Bit-Zeigerregister, oft auch mit Displacement.
R30/R31 = Z	Zeiger für Programmspeicherzugriff mit LPM.

```
.include "m328pdef.inc"
.def tmp = r16
.equ LED = 7

.org 0x0000
    rjmp init

init:
    ldi tmp, LOW(RAMEND)
    out SPL, tmp
    ldi tmp, HIGH(RAMEND)
    out SPH, tmp
    sbi DDRD, LED        ; PD7 als Ausgang
main:
    sbi PORTD, LED       ; LED an
    cbi PORTD, LED       ; LED aus
    rjmp main
```

- ★ Wenn du Assembler erklärst, trenne sauber: Quellcode mit Mnemonics -> Assembler erzeugt Maschinencode -> Mikrocontroller führt Maschinenbefehle aus.

7. AVR-I/O-Ports: DDRx, PORTx, PINx

AVR-Ports sind über Register organisiert. Ein Port besteht nicht nur aus einem Ausgang, sondern aus mehreren Speicherstellen für Richtung, Ausgangswert/Pull-up und Eingangszustand.

Register	Rolle	Beispiel
DDRx	Data Direction Register: 1 = Ausgang, 0 = Eingang.	sbi DDRD,7 macht PD7 zum Ausgang.
PORTx	Bei Ausgang: schreibt HIGH/LOW. Bei Eingang: 1 aktiviert internen Pull-up.	sbi PORTB,4 aktiviert Pull-up, wenn PB4 Eingang ist.
PINx	Leseregister für den tatsächlichen Zustand der Pins.	sbis PINB,4 prüft, ob PB4 gesetzt ist.

Wichtige I/O-Befehle

Befehl	Bedeutung
OUT port, reg	Schreibt ein Register in ein I/O-Register.
IN reg, port	Liest ein I/O-Register in ein Register.
SBI port, bit	Setzt ein einzelnes Bit im I/O-Space.
CBI port, bit	Löscht ein einzelnes Bit im I/O-Space.
SBIS reg, bit	Skip next instruction if bit is set.
SBIC reg, bit	Skip next instruction if bit is cleared.
LDI reg, k	Lädt eine Konstante; nur für R16-R31.

Pull-up/Pull-down und Taster

Ein offener Taster liefert ohne Widerstand keinen definierten Pegel. Pull-up oder Pull-down sorgen dafür, dass der Pin auch bei offenem Schalter eindeutig HIGH oder LOW ist. AVR bietet interne Pull-ups: Pin als Eingang, dann PORT-Bit auf 1.

```
; Taster an PB4 mit internem Pull-up, LED an PD7
sbi DDRD, 7      ; LED-Pin als Ausgang
cbi DDRB, 4      ; Taster-Pin als Eingang
sbi PORTB, 4     ; internen Pull-up aktivieren

main:
    sbis PINB, 4  ; wenn PB4 = 1, nächste Zeile überspringen
    sbi PORTD, 7  ; Taster gedrückt (LOW-aktiv) -> LED an
    sbic PINB, 4  ; wenn PB4 = 0, nächste Zeile überspringen
    cbi PORTD, 7  ; Taster losgelassen -> LED aus
    rjmp main
```



Bei LOW-aktiv bedeutet gedrückt/aktiv = 0. Dadurch muss die Programmlogik oft umgedreht werden.

8. Interrupts, Stack & Unterprogramme

Interrupts unterbrechen den normalen Programmablauf, damit auf Ereignisse schnell reagiert werden kann. Statt ständig zu prüfen, ob etwas passiert ist, springt der Mikrocontroller automatisch in eine Interrupt Service Routine.

Ablauf eines AVR-Interrupts

17. Aktueller Befehl wird beendet.
18. Globales I-Bit wird gelöscht, damit keine normalen verschachtelten Interrupts auftreten.
19. Program Counter wird auf dem Stack gesichert.
20. Adresse des Interruptvektors wird in den PC geladen.
21. ISR wird abgearbeitet.
22. RETI holt Rücksprungadresse vom Stack und setzt das I-Bit wieder.

Voraussetzungen

Voraussetzung	Warum wichtig?
Stack initialisieren	Rücksprungadressen von CALL/Interrupt müssen sicher abgelegt werden.
Interruptquelle konfigurieren	z. B. INT0-Flanke, Timer-Overflow, USART-Transmit.
Interrupt maskieren/freigeben	z. B. EIMSK, TIMSK, UCSR0B.
Globale Freigabe mit SEI	Erst dadurch werden maskierte Interrupts insgesamt erlaubt.
SREG sichern	Statusregister wird nicht automatisch gesichert; ISR kann Flags verändern.

Stack

Der Stack liegt im SRAM, beginnt typischerweise bei RAMEND und wächst nach unten. PUSH verringert den Stackpointer, POP erhöht ihn. CALL und Interrupts sichern Rücksprungadressen auf dem Stack.

```
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r16, HIGH(RAMEND)
out SPH, r16

; In einer ISR üblich:
in r15, SREG
push r16
; ... ISR-Code ...
pop r16
out SREG, r15
reti
```

Unterprogramme und Makros

Konzept	Eigenschaft
Unterprogramm	Aufruf mit RCALL/ICALL/CALL; Rückkehr mit RET; Code liegt einmal im Speicher.
Makro	Assembler ersetzt den Makroaufruf durch den Makrokörper; schneller, aber mehr Programmspeicher bei vielen Aufrufen.
Parameter	Können über Register, Stack, Speicher oder Makro-Dummies übergeben werden.

- ★ Bei Interruptfragen immer Stack + Vektortabelle + ISR + RETI + SREG erwähnen. Das ist meistens der Kern der Antwort.

9. Zeitsteuerung, Timer & PWM

Zeitsteuerung kann per Warteschleife oder Timer erfolgen. Warteschleifen blockieren die CPU und hängen direkt vom Takt ab. Timer laufen unabhängig vom Hauptprogramm und können Interrupts auslösen.

Warteschleifen

```
loop:
    dec r18          ; 1 Takt
    brne loop        ; meist 2 Takte, solange Sprung ausgeführt wird
```

Bei 8 MHz und einer Schleife mit ungefähr 3 Takten ergeben sich etwa $8.000.000 / 3 = 2,67$ Millionen Schleifendurchläufe pro Sekunde. Für lange Wartezeiten reichen 8-Bit-Register nicht, daher nutzt man verschachtelte Schleifen oder 16-Bit-Register.

Timer0-Modi

Modus	WGM02	WGM01	WGM00	Idee
Normal	0	0	0	TCNT0 zählt bis 255 und läuft dann über; Overflow-Interrupt möglich.
CTC	0	1	0	Clear Timer on Compare Match: TCNT0 zählt bis OCR0A und wird zurückgesetzt.
Fast PWM	egal	1	1	Schnelle PWM mit fester Periode, gut für LED/Motor/DAC-artige Nutzung.
Phasenkorrekt	egal	0	1	Auf- und Abwärtszählen, symmetrischere PWM, gut für Motorsteuerung.

Timer-Berechnung

Timer-Tick = CPU-Takt / Prescaler. Benötigte Zählschritte = gewünschte Zeit * Timer-Tick. Bei Normal Mode ggf. Startwert = $256 - \text{Zählschritte (8-Bit)}$ oder $65536 - \text{Zählschritte (16-Bit)}$.

PWM

PWM verändert bei fester Grundfrequenz das Verhältnis von Ein- zu Ausschaltzeit. Die Spannung ist digital voll an oder voll aus, aber der Mittelwert wirkt z. B. bei LED-Helligkeit oder Motorleistung analog.

PWM-Modus	Zählweise	Typische Nutzung
Fast PWM	Aufwärtszählend.	LED dimmen, Motoren steuern, einfache D/A-Wandlung.
Phasenrichtige PWM	Auf- und abwärtszählend.	Motorsteuerung mit symmetrischerem Signal.
Phasen- und frequenzrichtige PWM	Auf- und abwärtszählend mit anderem Update-Zeitpunkt.	Frequenz und Phase besser kontrollierbar.

10. USART, ADC & EEPROM

USART

USART steht für Universal Synchronous/Asynchronous Receiver and Transmitter. Es dient zur seriellen Kommunikation, z. B. Debug-Ausgabe oder Maschine-Maschine-Kommunikation. Im asynchronen Betrieb gibt es keine gemeinsame Taktleitung, deshalb müssen Baudrate und Datenformat auf beiden Seiten passen.

Register	Aufgabe
UCSR0A	Status und Spezialmodi, z. B. Double Speed.
UCSR0B	Senden/Empfangen aktivieren, Interrupts aktivieren, 9. Datenbit.
UCSR0C	Datenformat: asynchron/synchron, Parität, Stoppbits, Datenbits.
UDR0	Datenregister zum Senden/Empfangen.
UBRR0H/UBRR0L	Baudratenregister.

 Baudratenformel (asynchron normal): $UBRR = \text{Systemtakt} / (16 * \text{Zielbaudrate}) - 1$. Der Fehler sollte unter etwa 1 % bleiben.

ADC und EEPROM

Thema	Bedeutung
ADC / A-D-Wandler	Wandelt analoge Spannungen in digitale Werte. Wichtig für Sensoren, Potentiometer und Messwerte.
EEPROM	Nichtflüchtiger Speicher für Konfigurationen, Melodien, Kalibrierdaten. Beim ATmega328P 1 kByte.
EEPROM-Schreiben	Deutlich langsamer als RAM; Schreibzyklen begrenzt. Ablauf: warten, Adresse setzen, Daten setzen, Freigabebits korrekt setzen.
Power-Down	Vor dem Schlafmodus prüfen, ob ein EEPROM-Schreiben noch läuft, weil sonst der Oszillator ggf. nicht abgeschaltet wird.

I2C

I2C ist eine serielle, halbduplexe Schnittstelle mit aufwendigerem Protokoll als UART. Typisch sind zwei Leitungen: SCL für den Takt und SDA für Daten. Es eignet sich für Sensoren, Displays und Erweiterungsbausteine auf kurzen Strecken.

11. Intel 8086 / x86-Grundlagen

Der Intel 8086 ist ein 16-Bit-CISC-Mikroprozessor. Er kann bis zu 1 MB Speicher adressieren, nutzt Segmentierung und arbeitet intern mit BIU und EU. Die BIU kümmert sich um Speicher/Peripherie und Prefetch, die EU führt Befehle aus.

Assembler allgemein

Begriff	Erklärung
Assembler	Software, die Assembler-Code in Maschinensprache übersetzt.
Assemblersprache	Maschinennahe Sprache mit Mnemonics; für jede Plattform verschieden.
Mnemonic	Einprägsamer Befehlsname, z. B. MOV, ADD, INT.
Maschinencode	Binäre Befehle, die die CPU direkt ausführt.

Warum Assembler?

- **Kleine Programme:** wenig Overhead, direkter Hardwarezugriff.
- **Schnelle Programme:** gezielte Nutzung von Registern und CPU-Befehlen.
- **Volle Kontrolle:** alles nutzbar, was Hardware und Befehlssatz hergeben.
- **Verständnis:** zeigt, wie Hochsprachen am Ende tatsächlich auf Hardware ablaufen.

Programmstruktur unter DOS/TASM

```
.MODEL small
.DATA
    message db "Hallo Welt!$"
.CODE
    start:
        mov ax, @data
        mov ds, ax
        mov ah, 09h
        lea dx, message
        int 21h
        mov ah, 4Ch
        int 21h
    END start
```



Bei x86-Assembler immer zwischen Direktiven (.MODEL, .DATA, .CODE, END) und echten CPU-Befehlen (MOV, INT, ADD, JMP) unterscheiden.

12. x86-Register, Flags & Segmentierung

8086-Registergruppen

Gruppe	Register	Aufgabe
Datenregister	AX, BX, CX, DX	Allgemeine Daten. AX oft Akkumulator, BX für Adressen, CX als Zähler, DX bei Multiplikation/Division und I/O.
Segmentregister	CS, DS, SS, ES	Startadressen von Code-, Daten-, Stack- und Extra-Segment.
Zeigerregister	IP, SP, BP	IP zeigt auf nächsten Befehl, SP auf Stackspitze, BP für Stackzugriffe/Parameter.
Indexregister	SI, DI	Quell- und Zielindex, besonders bei String-/Speicherooperationen.
Flagregister	Status- und Control-Flags	Ergebnisinformationen und CPU-Arbeitsmodus.

Wichtige Flags

Flag	Name	Bedeutung
CF	Carry Flag	Übertrag bei unsigned Operationen oder Borrow bei Subtraktion.
PF	Parity Flag	Parität im niederwertigen Byte, gesetzt bei gerader Anzahl Einsen.
AF	Auxiliary Flag	Hilfsübertrag zwischen Nibbles, wichtig bei BCD-Arithmetik.
ZF	Zero Flag	Ergebnis ist 0; wichtig für JE/JZ und JNE/JNZ.
SF	Sign Flag	Vorzeichenbit des Ergebnisses ist gesetzt.
OF	Overflow Flag	Überlauf bei vorzeichenbehafteter Rechnung.
TF/IF/DF	Control Flags	Einzelschritt, Interruptfreigabe, Richtung bei Stringoperationen.

Segmentierung und Adressrechnung

Der 8086 besitzt einen 20-Bit-Adressbus und kann 1 MB adressieren. Eine physische Adresse entsteht aus Segmentbasis und Offset. Die Segmentbasis wird um 4 Bit nach links verschoben, also mit 16 multipliziert.

```
physische Adresse = Segment * 16 + Offset
```

Beispiel:

```
Segment = 0456h
```

```
Offset = 0123h
```

```
0456h * 10h = 04560h
```

```
04560h + 0123h = 04683h
```

Segment	Typischer Offset
CS - Code Segment	IP - Instruction Pointer.
DS - Data Segment	BX, SI, DI oder direkte Adresse im Befehl.
SS - Stack Segment	SP oder BP.

ES - Extra Segment	Zusätzlicher Datenbereich, oft bei Stringoperationen.
--------------------	---

 Segmentierung ermöglicht verschiebbare Programme: Ändert man das Segmentregister, können Code/Daten an anderer physischer Stelle liegen, ohne alle Offsets neu zu berechnen.

13. DOS-Interrupts, Schleifen, Prozeduren & Makros

DOS-Interrupt 21h

INT 21h ist ein Softwareinterrupt und dient als DOS-Funktions-Dispatcher. Die gewünschte Funktion steht meist in AH, Parameter und Rückgabewerte liegen in Registern.

Funktion	Bedeutung
AH=01h	Tastatureingabe mit Echo, Zeichen in AL.
AH=02h	Zeichen aus DL auf Bildschirm ausgeben.
AH=08h	Tastatureingabe ohne Echo, Zeichen in AL.
AH=09h	Zeichenkette ab DS:DX ausgeben; Ende mit \$.
AH=0Ah	Zeichenkette einlesen; Pufferadresse in DS:DX.
AH=4Ch	Programm sauber beenden und Kontrolle an DOS zurückgeben.

Sprünge und Schleifen

Bedingte Sprünge reagieren auf Flags, die z. B. durch CMP oder arithmetische Befehle gesetzt wurden.

Sprung	Bedeutung
JE / JZ	Jump if equal / zero - Ergebnis/Vergleich ist 0.
JNE / JNZ	Jump if not equal / not zero.
JA / JAE	Unsigned: above / above or equal.
JB / JBE	Unsigned: below / below or equal.
JC / JNC	Carry gesetzt / nicht gesetzt.

Prozeduren, Stack und Parameter

Prozeduren vermeiden doppelte Codeblöcke. CALL speichert die Rücksprungadresse auf dem Stack, RET kehrt zurück. Parameter können über Register, Stack oder Speicher übergeben werden. Der Base Pointer BP eignet sich, um Parameter auf dem Stack stabil zu adressieren.

Parameterübergabe	Vorteil	Nachteil
Register	Sehr schnell und einfach.	Nur wenige Parameter möglich, Registerbelegung muss bekannt sein.
Stack / Call by Value	Gut strukturiert, verschachtelte Aufrufe möglich.	Mehr Speicherzugriffe und sauberer Stackaufbau nötig.
Stack / Call by Reference	Große Daten müssen nicht kopiert werden.	Aufgerufene Prozedur kann Originaldaten verändern.
Globale Variablen	Einfach erreichbar.	Fehleranfällig, schlecht wiederverwendbar, nicht mehr zeitgemäß.

Makros vs. Prozeduren

Kriterium	Makro	Prozedur
Speicher	Makrokörper wird bei jedem Aufruf eingefügt - Programm kann wachsen.	Code liegt einmal im Speicher.
Laufzeit	Kein CALL/RET, daher oft schneller.	CALL/RET und Stackzugriffe kosten Zeit.
Einsatz	Kleine, häufige Quellcodebausteine.	Größere wiederverwendbare Funktionen.
Besonderheit	Lokale Sprungmarken mit LOCAL verwenden.	Register sichern, Parameterübergabe festlegen.



Bei Stack-Fragen immer die Reihenfolge nennen: Was zuletzt gepusht wurde, wird zuerst gepoppt. PUSH/POP müssen paarweise und in umgekehrter Reihenfolge passen.

14. Prüfungsfragen & Kurz-Wiederholung

Dieser Abschnitt ist als schnelle Wiederholung direkt vor der mündlichen Prüfung gedacht. Ziel ist nicht, alles auswendig zu lernen, sondern zu jedem Kernbegriff eine klare Antwortstruktur zu haben.

Kompakte Prüfungsfragen

Frage	Antwort
Was macht die ALU?	Sie führt einfache arithmetische und logische Operationen aus und setzt dabei Flags für Sprünge oder Statusabfragen.
Unterschied Mikroprozessor / Mikrocontroller?	Mikroprozessor = CPU auf einem Chip. Mikrocontroller = CPU + Speicher + Peripherie (Timer, ADC, UART, I/O).
DDRx, PORTx und PINx beim AVR?	DDRx stellt die Richtung ein, PORTx schreibt Ausgangswerte oder aktiviert Pull-ups, PINx liest den realen Pinzustand.
Warum braucht man Pull-ups?	Damit ein offener Taster einen definierten Pegel liefert und der Eingang nicht durch Störungen zufällig kippt.
Was passiert bei einem Interrupt?	Befehl endet, PC wird auf dem Stack gesichert, ISR-Adresse wird geladen, ISR läuft, RETI kehrt zurück.

Warum muss der Stack initialisiert werden?	CALL, RET, Interrupts und PUSH/POP brauchen einen gültigen Speicherbereich für Rücksprungadressen.
Timer vs. Warteschleife?	Warteschleife blockiert die CPU. Timer zählt unabhängig und kann Interrupts oder Ausgangssignale erzeugen.
Was macht PWM?	Sie verändert das Tastverhältnis, sodass der Mittelwert z. B. LED-Helligkeit oder Motorleistung beeinflusst.
Physische Adresse beim 8086?	$\text{Segment} * 16 + \text{Offset} = \text{physische Adresse}$.
Was ist INT 21h?	Ein DOS-Softwareinterrupt/Funktionsdispatcher für Ein-/Ausgabe, Dateifunktionen und Programmende.

Formeln und Mini-Merksätze

Thema	Merksatz / Formel
8086-Adresse	$\text{physische Adresse} = \text{Segment} * 16 + \text{Offset}$.
AVR-Timer	$\text{Timer-Tick} = \text{CPU-Takt} / \text{Prescaler}$; $\text{Zählschritte} = \text{Zeit} * \text{Timer-Tick}$.
UART Baudrate	$\text{UBRR} = f_{\text{CPU}} / (16 * \text{Baudrate}) - 1$ im normalen asynchronen Modus.
Stack	LIFO: Last In, First Out. PUSH runter, POP rauf.
LOW-aktiv	Aktiv bedeutet 0; Logik im Code entsprechend umdrehen.
Makro vs. Prozedur	Makro schneller/größer, Prozedur kleiner/strukturierter.
RISC vs. CISC	RISC: einfache regelmäßige Befehle. CISC: komplexere historisch gewachsene Befehle.

★ Übe die Antworten laut: 'Was ist es? Wofür braucht man es? Welches Register/Befehl/Beispiel gehört dazu?' Diese Dreierstruktur passt fast immer.

Viel Erfolg bei der mündlichen Prüfung! 🎓